

LAPORAN TUGAS AKHIR (UAS)
STRUKTUR DATA DAN ALGORITMA: SISTEM MANAJEMEN AKADEMIK
SISWA



Disusun Oleh:

KELOMPOK 2

NAMA	NIM
Arsy Agung Nurmawan	2503010137
Fabian Taufiqurrahman	2503010004
Raka Brawijaya	2503010137
Randhan Hidayat	2503010001

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS KELOMPOK 2

2026

1. Daftar Anggota Kelompok

Proyek ini dikembangkan secara kolaboratif oleh Kelompok 2 dengan rincian tugas per anggota sebagai berikut:

- **Fabian Taufiqurrahman:** Penanggung jawab visualisasi data, pembuatan fungsi cetak data, dan desain tabel output.

- **Randhan Hidayat:** Penanggung jawab perancangan algoritma pengurutan data (*sorting algorithm*).
- **Raka Brawijaya:** Penanggung jawab perancangan algoritma pencarian data (*searching algorithm*).
- **Arsy Agung Nurmawan:** Penanggung jawab analisis data awal (dummy data) dan penyusunan dokumentasi/laporan.

2. Studi Kasus yang Dipilih

Kelompok kami memilih studi kasus "**Sistem Informasi Manajemen Nilai Akademik Siswa Berbasis Komputasi Terstruktur**".

Latar Belakang Masalah:

Di banyak institusi pendidikan berskala kecil atau menengah, pencatatan nilai siswa kerap mengalami kendala saat volume data mulai meningkat. Tanpa sistem yang terstruktur, petugas administrasi atau guru akan kesulitan untuk:

1. Melihat urutan siswa secara teratur berdasarkan Nomor Induk Siswa (NIS).
2. Melakukan pencarian data nilai siswa secara cepat ketika dibutuhkan dalam rapat evaluasi akademik.

Solusi yang Ditawarkan: Aplikasi berbasis C++ ini dirancang untuk mengotomatisasi penyimpanan records akademik siswa menggunakan kombinasi struktur data dinamis (`std::vector`) dan tipe data bentukan (`struct`). Sistem ini dilengkapi fungsi pengurutan otomatis dan pencarian instan untuk memotong waktu administrasi sekolah.

3. Analisis Kebutuhan Sistem

A. Kebutuhan Fungsional (*Functional Requirements*)

- **Sistem Pemuatan Data Awal:** Aplikasi harus otomatis memuat 4 data dummy anggota kelompok saat pertama kali dijalankan sebagai basis pengujian awal.
- **Sistem Input Dinamis:** Pengguna dapat menambahkan data siswa baru tanpa batasan kuota di awal (menginput NIS, Nama, Kelas, Mata Pelajaran, dan Nilai).
- **Sistem Visualisasi Tabel:** Menyajikan seluruh data secara tabular terstruktur yang rapi dengan ukuran kolom yang konsisten.
- **Sistem Sortir Data:** Mengurutkan tumpukan data akademik berdasarkan nomor NIS secara menaik (*ascending*).
- **Sistem Cari Data:** Menemukan record nilai siswa secara spesifik berdasarkan kata kunci NIS.

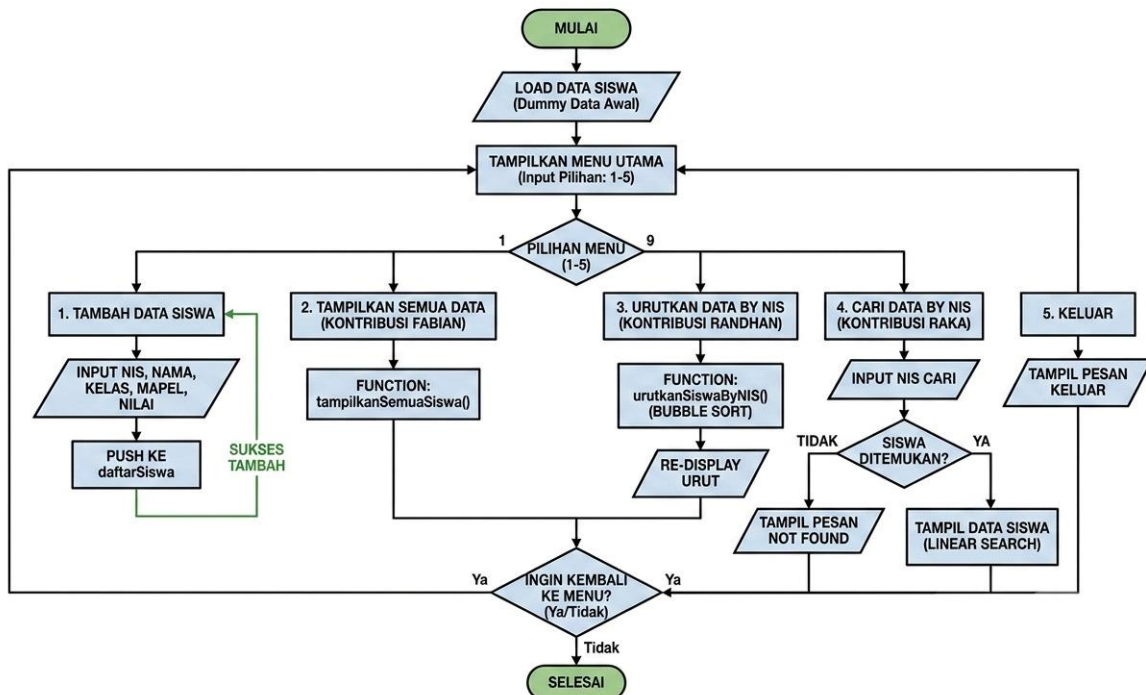
B. Kebutuhan Non-Fungsional (*Non-Functional Requirements*)

- **Kompatibilitas Mundur (Backward Compatibility):** Menggunakan standar penulisan C++98 agar program dapat dikompilasi dengan lancar tanpa *error* di IDE lama seperti Dev-C++ versi lawas.
- **Robustness (Ketahanan Input):** Menyediakan mekanisme pembersihan memori buffer (`cin.ignore()`) agar pembacaan teks panjang yang mengandung spasi (`getline`) tidak mengalami kegagalan *skipping*.

4. Desain Sistem

Alur eksekusi aplikasi dirancang menggunakan arsitektur *Menu-Driven Event Loop* dengan diagram aktivitas sebagai berikut:

SISTEM AKADEMIK KELOMPOK 2 (Case Study 12)



5. Struktur Data yang Digunakan

Implementasi kode pada file PROJECT-UAS.cpp menerapkan dua jenis penanganan struktur data:

A. Tipe Data Bentukan (struct)

Digunakan untuk merepresentasikan satu entitas objek Siswa yang mengikat 5 atribut berbeda (heterogen) menjadi satu kesatuan unit data:

C++

```
struct Siswa {
    string nis;
    string nama;
    string kelas;
    string mataPelajaran;
    int nilai;
};
```

B. Kontainer Dinamis (std::vector)

Sebagai media penyimpanan utama (*database temporary*) di dalam RAM yang bertugas menampung jajaran objek Siswa:

```
C++
```

```
vector<Siswa> daftarSiswa;
```

6. Alasan Pemilihan Struktur Data

1. **Efisiensi Manipulasi Memori (struct):** Jika tidak menggunakan struct, kita harus membuat 5 array terpisah untuk masing-masing atribut. Ketika algoritma *sorting* menukar posisi data, kita harus menukar 5 array tersebut secara manual satu per satu. Dengan struct, penukaran tempat (*swapping*) cukup dilakukan sekali pada objek Siswa terkait, sehingga instruksi CPU menjadi lebih hemat.
2. **Fleksibilitas Alokasi (vector):** Berbeda dengan Array Statis (Siswa daftar[100]) yang membuang-buang ruang memori jika data sedikit dan *crash* jika data melebihi 100, `std::vector` akan mengalokasikan ruang memori tambahan secara dinamis sesuai kebutuhan riil data yang dimasukkan melalui fungsi `push_back()`.

7. Pembagian Tugas Anggota

Secara detail mekanis, kontribusi tim terbagi ke dalam modularitas fungsi di dalam file PROJECT-UAS.cpp:

A. Modul Tampil Data (Fabian Taufiqurrahman)

Bertanggung jawab atas visualisasi data dengan memanfaatkan pustaka `<iomanip>`. Menggunakan `setw()` untuk mengunci lebar kolom dan `setfill()` untuk membuat garis pembatas tabel yang presisi.

```
C++
```

```
void tampilkanHeaderTabel();
```

```
void tampilkanSemuaSiswa(const vector<Siswa>& daftarSiswa);
```

B. Modul Algoritma Sorting (Randhan Hidayat)

Mengimplementasikan **Algoritma Bubble Sort**. Melakukan iterasi bersarang (*nested loop*) untuk membandingkan string nis. Jika ditemukan nilai indeks ke- j lebih besar dari indeks ke- $j+1$, dilakukan operasi penukaran tempat (*swap*) elemen.

```
C++
```

```
void urutkanSiswaByNIS(vector<Siswa>& daftarSiswa);
```

C. Modul Algoritma Searching (Raka Brawijaya)

Mengimplementasikan **Algoritma Linear Search**. Melakukan pemindaian satu per satu (*sequential*) dari indeks awal 0 hingga ukuran maksimal vector. Mengembalikan nilai indeks jika ditemukan, dan mengembalikan -1 jika data nihil.

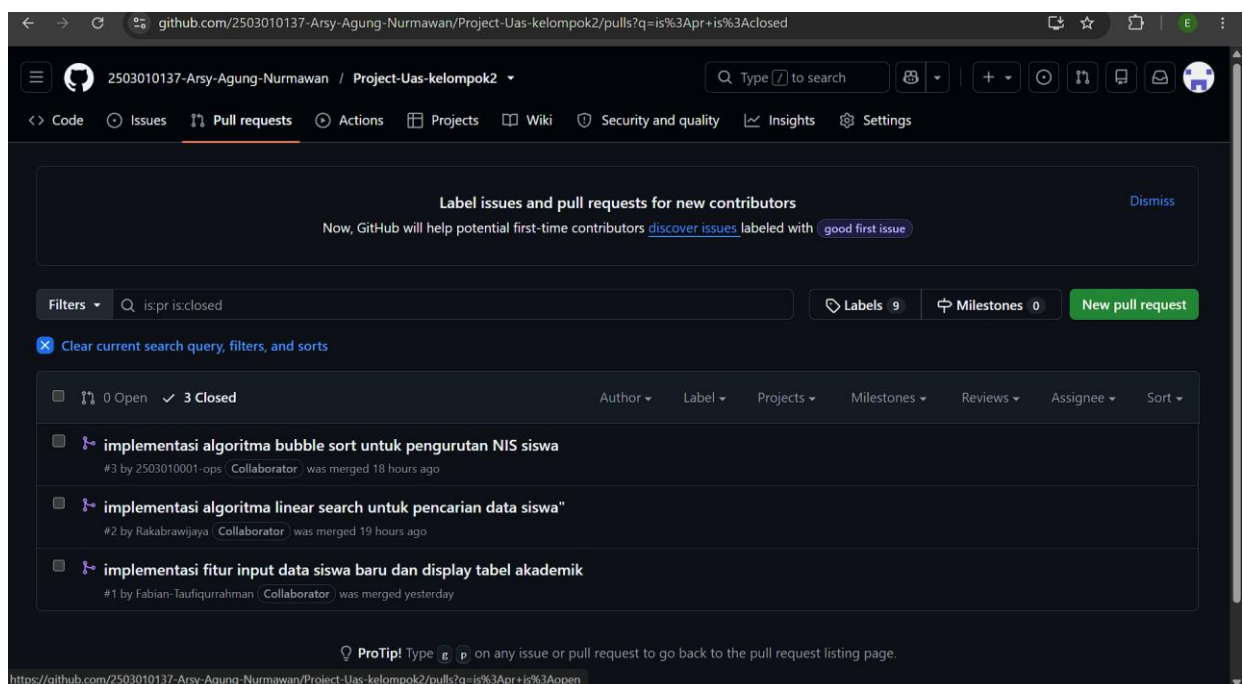
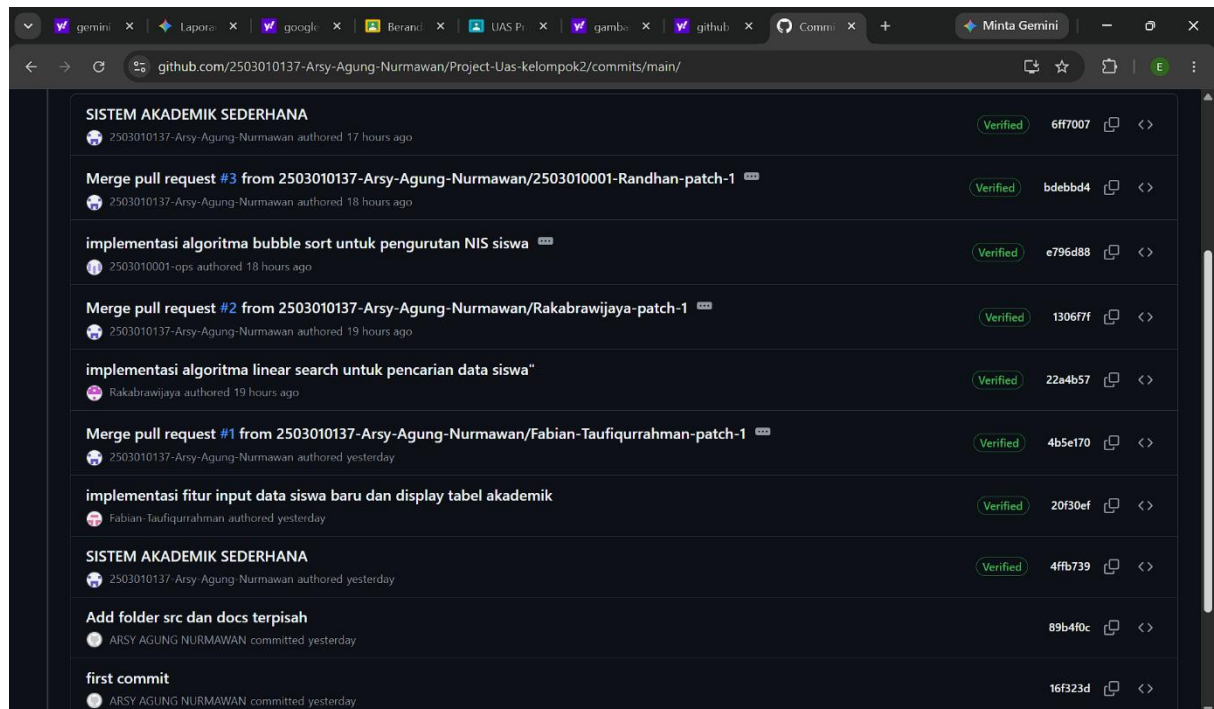
```
C++
```

```
int cariSiswaByNIS(const vector<Siswa>& daftarSiswa, string nisCari);
```

D. Modul Data Awal & Integrasi (Arsy Agung Nurmawan)

Mengintegrasikan seluruh fungsi ke dalam blok `int main()`, merancang interaksi menu lewat struktur `switch-case`, serta menyuntikkan data dummy awal dari profil anggota kelompok.

8. Screenshot Commit dan PullRequestGitHub



9. Kesimpulan

Dari hasil perancangan, pembuatan kode program PROJECT-UAS.cpp, hingga tahap pengujian sistem, Kelompok 2 menarik kesimpulan bahwa:

1. Penggabungan struktur data struct dan kontainer std::vector menghasilkan pengelolaan data nilai akademik yang ringkas, aman dari kebocoran memori (*memory leak*), dan sangat fleksibel untuk penambahan data dinamis.
2. **Algoritma Bubble Sort** yang diterapkan berhasil mengurutkan data NIS secara alfabetis/numerik dengan tepat, meskipun untuk pengembangan skala besar disarankan beralih ke *Quick Sort* atau *Merge Sort*.
3. **Algoritma Linear Search** berjalan sukses dan mampu menampilkan informasi detail data siswa secara presisi dengan kondisi penanganan eror yang baik jika data yang dicari tidak ditemukan di dalam sistem[cite: 1].
4. Pemanfaatan *GitHub Version Control* sangat krusial dalam membagi beban kerja tim secara modular, mempercepat proses *debugging*, serta melatih koordinasi kerja seperti tim pengembang perangkat lunak profesional.